Infor Login Events Visualizer

Global login activity dashboard — User & Developer Guide

Audit & Monitoring · Infor OS Cloud

Generated 2026-10-26 · applies to the login-visualizer app

The **Login Events Visualizer** is a web dashboard that shows user **LOGIN** and **LOGOUT** activity from an Infor OS tenant on an interactive world map, with KPIs, filters, localization, and themes. All location data comes directly from the Infor monitoring API — no external geolocation service is used.

Part 1 — User Guide

Opening the dashboard

Open the application URL in any modern browser. If the app has been idle (free hosting tiers sleep after inactivity), the first load may take 30–60 seconds while the service wakes; a refresh afterward is instant.

The header controls

- **Time period** — choose `1h`, `6h`, `24h`, `7d`, or `30d`. All panels update to that window.
- **User** — filter the whole dashboard to a single user, or leave on "All users".
- **Language** — switch the interface language. The app also auto-detects your browser language on first visit.
- **Theme** — Dark, Light, Ocean, or High contrast. Your choice is remembered on this device.
- **Refresh (↻)** — re-fetch the latest data for the current window.

KPI cards

- **Logins** — successful logins in the window (logouts shown beneath).
- **Distinct users** — unique users seen.
- **Countries** — number of distinct countries logins came from.
- **Avg session** — average session length (median shown beneath), computed from logout events.
- **MFA logins** — percentage of logins with multi-factor enabled.

- **Office / private IP** — logins from internal/non-routable network addresses (e.g. a corporate office network).

Map & charts

- **Login locations** — each marker is a city; larger/clustered markers mean more logins. Click a marker for city, country, login count, and user count.
- **Logins over time** — a timeline with a time axis; hover a bar for the exact time bucket and count.
- **Top countries / Top users** — ranked bars for the current window.

Recent events table

Lists the most recent events. You can:

- **Search** — type to filter by user, city, country, or IP.
- **Filter by action** — All / LOGIN / LOGOUT.
- **Sort** — click any column header to sort (click again to reverse).

Country names and dates/times are shown in your selected language and locale.

Part 2 — Developer Guide

What it is

A zero-dependency Node.js application (Node ≥ 18, uses the built-in `http` module and global `fetch`) that proxies the Infor monitoring API and serves a static Leaflet-based frontend. The only runtime input is a set of Infor API Gateway credentials (a `.ionapi` file).

Project layout

```
login-visualizer/
  server.js      HTTP server + API endpoints + static host + response cache
  infor.js       OAuth token mgmt + paginated monitoring log fetch
  transform.js   normalize records, compute KPIs, aggregate for charts/map
  public/
    index.html   dashboard markup
    styles.css   themes (dark/light/ocean/contrast) + layout
    app.js       data fetch + render (KPIs, map, charts, table) + filters/sort
    i18n.js      locale detection + Intl formatting + translations (EN/ES/FR/DE)
  render.yaml    Render Blueprint
  README.md, SETUP-AND-API-GUIDE.md
```

Credentials (how the app authenticates)

The app reads Infor credentials from one of two environment variables:

- `IONAPI_PATH` — filesystem path to a `.ionapi` file (local dev).
- `IONAPI_JSON` — the full `.ionapi` JSON inline on one line (used in hosting).

> **Never commit a `.ionapi` file.** It contains long-lived service-account secrets (`cs`, `saak`, `sask`). The repo `.gitignore` excludes it. Secrets stay server-side and are never sent to the browser.

The Infor API it calls

Everything goes through the API Gateway. The gateway base URL (`iu`) does **not** include the tenant segment, so paths are prefixed with the tenant id (`ti`):

```
POST {iu}/{ti}/AUDITMONITOR/monitoringreports/v3/logs
Header: x-monitoring-source: sts
Body:
{
  "fetchsize": 1000,
  "page": 1,
  "sortorder": "logTime DESC",
  "query": "logTime BETWEEN \"<from ISO+offset>\" TO \"<to ISO+offset>\" AND action IN
[\"LOGIN\",\"LOGOUT\"]",
  "timezone-offset": "00"
}
```

> **Result cap:** the monitoring API returns at most **10,000** results. The app paginates up to 10 pages of 1,000 and stops.

Response fields used

Field	Meaning
`action`	LOGIN or LOGOUT
`outcome`	SUCCESS / failure
`logTime`	Event time (ISO-8601 Z)
`principal.userName / fullName`	Who logged in
`sessionContext.sessionDuration`	Seconds (present on LOGOUT)
`authContext.mfa.enabled`	MFA used?
`connection.geoLocation`	country (2-letter ISO), city, lat, lon, timezone
`connection.ipAddresses`	Client IP (used for office/private detection)

Geolocation is embedded in the payload, so **no external geolocation API is called**. Country codes are converted to full names in the browser via `Intl.DisplayNames`; city names may be URL-encoded and are decoded for display.

HTTP endpoints the frontend uses

Endpoint	Purpose
`GET /api/events?period=&user=`	KPIs, aggregations, map points, recent events
`GET /api/users?period=`	Distinct users for the filter dropdown
`GET /api/health`	Liveness check

A 60-second in-memory cache (keyed by period) reduces load on the tenant.

Run locally

```
# PowerShell
$env:IONAPI_PATH = "C:\path\to\YOUR-TENANT.ionapi"
$env:PORT = "3000"
node server.js
# open http://localhost:3000
```

Deploy (Render)

- Render's Node runtime deploys **from a Git repo** (no file-upload path). Push the folder to a repo; `.gitignore` keeps secrets out.
- Create a Web Service (runtime Node, build `echo no build step`, start `node server.js`).
- Set a secret env var `IONAPI_JSON` = your `.ionapi` contents on one line.

> **Gotcha:** setting env vars via the Render *API* does not trigger a redeploy — the instance keeps the old environment. Edit the var in the dashboard (auto-redeploys) or POST a new deploy.

Switching tenants

Everything tenant-specific lives in the `.ionapi`. To target a different tenant, point `IONAPI_PATH` at another file (local) or replace the `IONAPI_JSON` value (hosted) and redeploy. No code changes.

Security

- HTTPS only; secrets stay server-side and are never returned to the browser.
- The app has **no built-in user authentication** — anyone with the URL sees the configured tenant's data. Keep URLs private or add an auth layer before exposing sensitive tenants.
- It uses one service-account credential, so it sees whatever that account can see tenant-wide. Per-user access control would require an OIDC redesign.
- Rotate any credential that is ever exposed.